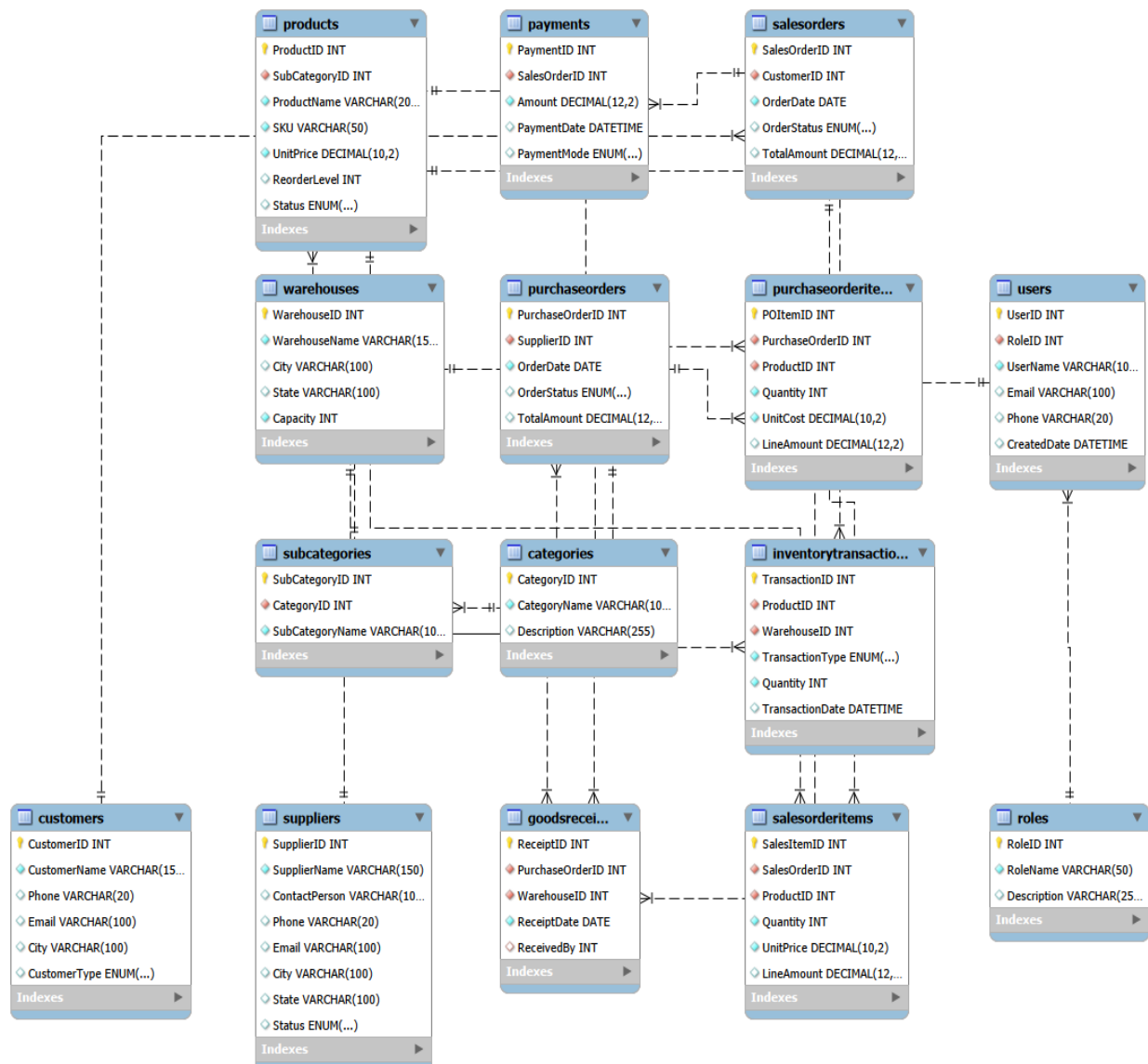


Inventory Management System

Name: Surekaa M

GitHub URL: https://github.com/surek96/Inventory_Management_system_mySql

Table Schema:



Basic Level:

1. Display all products.
2. Display all customers.
3. Find products above price 100.
4. List suppliers from Chicago.
5. Display customers ordered by name.
6. Count customers.
7. Count suppliers.
8. Show products by category.
9. Find Average, Lowest and Highest product price.
10. Top 5 Suppliers based on Total Purchase Value.
11. Show Revenue (total amount) generated by Customer Type.
12. Count orders.
13. Show bottom 10 customers who generated lowest revenue (total amount)
14. Find products with reorder level below 10.
15. How many sales orders are currently 'Pending', 'Shipped', or 'Cancelled'?
16. Customers from South James.
17. Products ordered alphabetically.
18. Retrieve a list of all active suppliers, showing their names, contact persons, and cities.
19. Total sales orders.
20. Product sales summary.

Intermediate Level:

21. List the ProductName, SKU, and UnitPrice for all products that cost more than the average unit price of all products.
22. Find the names of all Warehouses that have never had any transactions recorded in the InventoryTransactions table.
23. Find all Products that have never been ordered by a customer.
24. List the CustomerName and Phone of all customers who have placed at least one sales order.
25. Find the names of all Suppliers who have at least one pending purchase order.
26. Find the details of all SalesOrders that have a TotalAmount greater than the highest single payment ever made in the Payments table.
27. List all ProductName and SKU items that belong to the 'category_7' category.
28. Rank products within each subcategory based on their total sales quantity. Show the top 3 highest-selling products for every single subcategory.
29. For each CustomerType, rank customers based on their lifetime TotalAmount spent.

30. For each customer, list their orders alongside the date of their previous order and calculate the number of days that passed between the two orders. (Hint: Use the LAG() function).
31. Write a query using a CTE to calculate the total item count and total line amount for each order line in SalesOrderItems. In your main query, filter the result to only show order lines where the total calculated line amount is greater than \$1,000.
32. Create a CTE that joins SalesOrders with Customers to create a flat list of SalesOrderID, OrderDate, TotalAmount, and CustomerName. In the main query, select from this CTE to find all orders placed in the current year.
33. Use a CTE to pull a simple list of SubCategoryID, SubCategoryName, and its parent CategoryName (by joining those two tables). In your main query, select from this CTE to list all subcategories that belong to the 'category_3' category.
34. Create a CTE that pulls a unique list of all ProductIDs present in the InventoryTransactions table. In your outer query, select all product names from the Products table whose ID is found in that CTE.
35. Create a CTE called ActiveSuppliers that filters the Suppliers table for only Chicago city with a status of 'Active'. In your main query, pull from this CTE to extract a mailing list showing just the SupplierName, ContactPerson, and Email.

Advanced Level:

36. Write a query to create a view named v_ProductCatalog that combines the products, subcategories, and categories tables. This view should display the product ID, name, SKU, unit price, subcategory name, and main category name for easy searching.
37. Create a view called v_SalesOrderSummary that joins sales orders, customers, and payments to show an overview of every order. It needs to display the order ID, customer name, order status, total amount, and the total amount paid so far.
38. Write a trigger named trgSetCreatedDate that fires automatically before a new user is inserted into the Users table. The trigger should set the CreatedDate field to the current system date and time if it is left blank.
39. Write a query that selects SupplierName and uses a CASE statement to create a new column called AccountStatus. If Status is 'Active', show "Fully Approved". If Status is 'Inactive', show "On Hold". For any other value, show "Under Review".
40. Write a query for the Customers table that outputs CustomerName alongside a column named ContactCompleteness. If both Email AND Phone are missing (IS NULL), label them as "No Contact Info". If only one is missing, label them as "Partial Info". If both are present, label them as "Complete Profile".
41. Write a query on the PurchaseOrders table that selects PurchaseOrderID and OrderStatus. Add a calculated column named RequiresFollowUp. If OrderStatus is 'Pending', display 'Yes'. If it is anything else, display 'No'.

42. Query the Products table to list ProductName and UnitPrice. Create a new column named PriceTier. If UnitPrice is greater than 500, label it 'Premium'. If it is less than or equal to ₹1,00, label it 'Standard'.
43. Write a query on the SalesOrders table displaying SalesOrderID and TotalAmount. Add an audit column called InsuranceRequired. If TotalAmount is greater than ₹5000, return the number 1 (or 'Required'). Otherwise, return the number 0 (or 'Not Required').